Where Deep Learning Losses Come From

Likelihood, cross-entropy, and regularization from probability

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Three familiar losses, one probabilistic origin

You have already met these losses:

$$\mathcal{L}_{\text{reg}} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \quad \text{(regression)}$$

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)] \quad \text{(binary classification)}$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda\|\boldsymbol{\theta}\|_2^2 \quad \text{(regularized)}$$

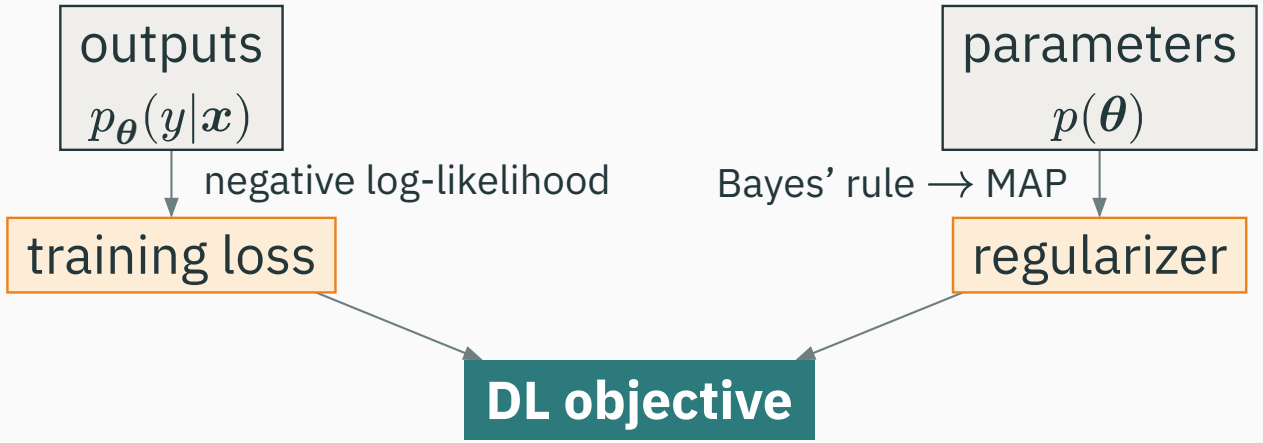
Objective. Derive each term from an explicit likelihood or prior.

The map for today

V VISUAL

Two probabilistic assumptions determine the two halves of the objective.

We will define the predictive distribution $p_{\boldsymbol{\theta}}(y|\boldsymbol{x})$, likelihood, log-likelihood, and NLL; then the prior $p(\boldsymbol{\theta})$ and MAP.



Probability primer

A point prediction summarizes a distribution

V VISUAL

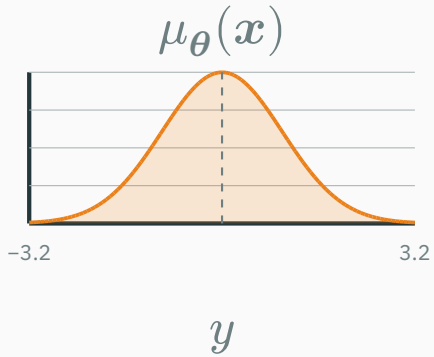
REGRESSION · $Y \in \mathbb{R}$

Point prediction: mean of $Y \mid \boldsymbol{X} = \boldsymbol{x}$

$$\hat{y}(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{\theta}}[Y \mid \boldsymbol{X} = \boldsymbol{x}] = \mu_{\boldsymbol{\theta}}(\boldsymbol{x})$$

Full predictive distribution

$$Y \mid \boldsymbol{X} = \boldsymbol{x} \sim \mathcal{N}\big(\mu_{\boldsymbol{\theta}}(\boldsymbol{x}), \sigma^2_{\boldsymbol{\theta}}(\boldsymbol{x})\big)$$



BINARY CLASSIFICATION · $Y \in \{0, 1\}$

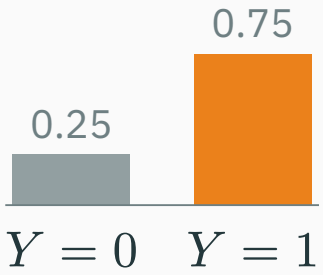
Point prediction: mode of $Y \mid \boldsymbol{X} = \boldsymbol{x}$

$$\hat{y}(\boldsymbol{x}) = \mathbf{1}[p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq 0.5]$$

Full predictive distribution

$$Y \mid \boldsymbol{X} = \boldsymbol{x} \sim \text{Bernoulli}(p_{\boldsymbol{\theta}}(\boldsymbol{x}))$$

example: $p_{\boldsymbol{\theta}}(\boldsymbol{x}) = 0.75$



Neural network: $x \to$ distribution parameters. Prediction: $\hat{y}$ = mean or mode.

Discrete distributions assign probability to outcomes

V VISUAL

The **support** $\mathcal{S}_Y$ is the set of values that Y can take.

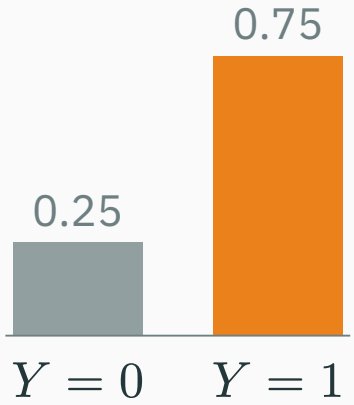
Probability mass function (PMF)

$$p_{Y(y)} = P(Y = y), \quad y \in \mathcal{S}_Y$$

$$p_{Y(y)} \geq 0, \quad \sum_{y \in \mathcal{S}_Y} p_{Y(y)} = 1$$

Examples. Bernoulli: $\{0, 1\}$ · categorical: $\{1, ..., K\}$ · Poisson: $\{0, 1, 2, ...\}$

Bernoulli(0.75) PMF



each bar is an outcome probability

Exact values can have nonzero probability: here $P(Y = 1) = 0.75$.

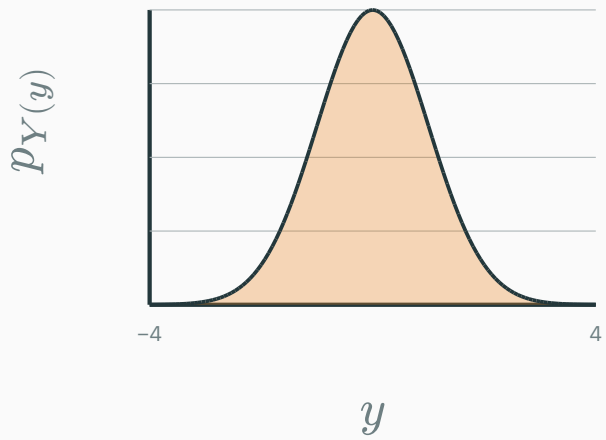
Continuous distributions assign density across a support

V VISUAL

The **support** may be bounded or unbounded:

$\mathcal{S}_Y = [a, b]$

for a Uniform variable; $\mathcal{S}_Y = \mathbb{R}$ for a Gaussian.



Probability density function (PDF)

$$p_{Y(y)} \geq 0, \quad \int_{\mathcal{S}_Y} p_{Y(y)} \, \mathrm{d}y = 1$$

the total area under a PDF is 1

For continuous Y, probability comes from area under the density.

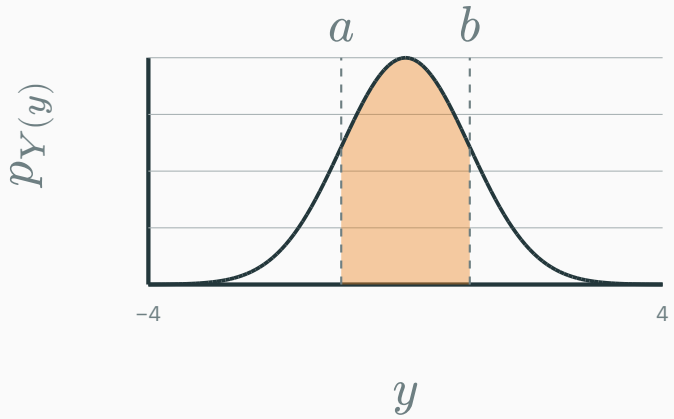
Density is not probability

V VISUAL

For a continuous Y, probability comes from **area**:

$$P(a \leq Y \leq b) = \int_a^b p_{Y(y)} \, \mathrm{d}y$$

$p_{Y(y)} \neq P(Y = y)$ — a density can **exceed 1**; only *integrals* are probabilities.



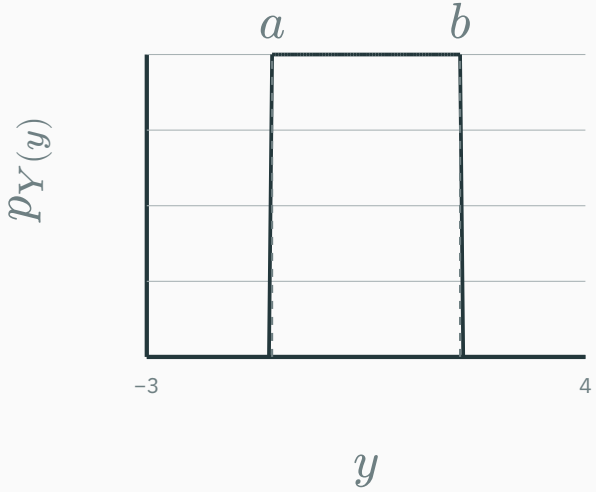
the shaded area is $P(a \leq Y \leq b)$

Uniform distribution: support matters

V VISUAL

$$Y \sim \mathcal{U}(a, b) \quad p_{Y(y)} = \begin{cases} 1/(b-a) & a \leq y \leq b \\ 0 & \text{otherwise} \end{cases}$$

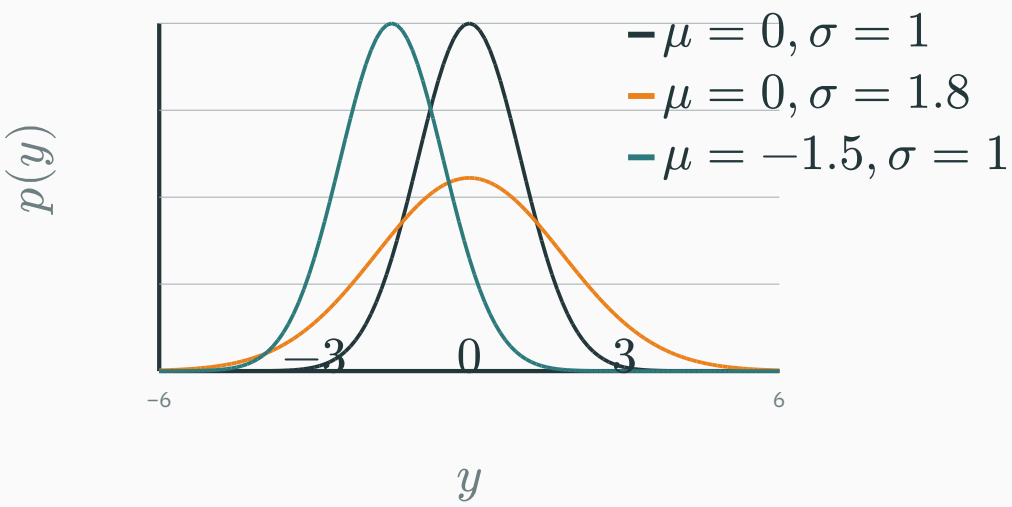
The support matters: outside $[a, b]$, the density is exactly 0.



Gaussian distribution

V VISUAL

$$Y \sim \mathcal{N}(\mu, \sigma^2) \qquad p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)$$



Only two knobs: **location** μ and **scale** σ.

From probability to likelihood

Read the model notation one symbol at a time

Q QUESTION

$$p_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$$

$\boldsymbol{X}$ / $\boldsymbol{x}$	$\boldsymbol{X}$ is the random input vector; $\boldsymbol{x}$ is one observed input
Y / y	Y is the random target; y is one possible or observed value
$\boldsymbol{\theta}$	the **parameter vector** — the model's learned weights and biases
$p_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$	the probability (discrete Y) or density (continuous Y) assigned to y, **given** $\boldsymbol{x}$

Read aloud: "model θ's probability or density for y, given x."

Prediction fixes the model and varies the outcome

V VISUAL

Example: $Y \in \{0, 1\}$. Fix the input $x^\star$ and one parameter setting θ_A.

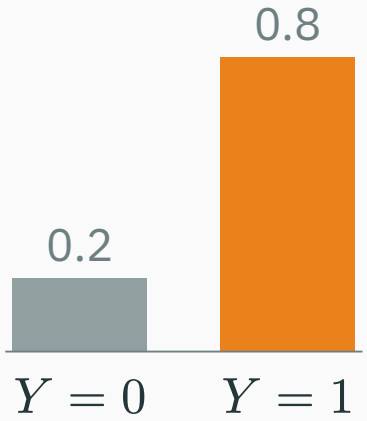
$$p_{\theta_A}(Y = 0 \mid x^\star) = 0.20$$

$$p_{\theta_A}(Y = 1 \mid x^\star) = 0.80$$

$$0.20 + 0.80 = 1$$

We vary the possible value of Y while keeping θ_A and $x^\star$ fixed.

predictive distribution



$$\hat{y}(x^\star) = \arg\max_y p_{\theta_A}(y \mid x^\star) = 1$$

Likelihood fixes data and varies the model

D DERIVATION

Now observe $y^\star = 1$ for the input $x^\star$, so $\mathcal{D} = \{(x^\star, 1)\}$.

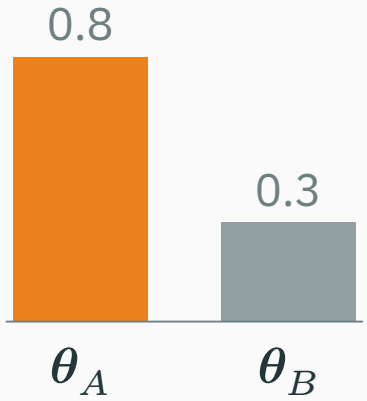
$$L(\theta; \mathcal{D}) = p_\theta(y^\star = 1 \mid x^\star)$$

Candidate A: $L(\theta_A; \mathcal{D}) = 0.80$
Candidate B: $L(\theta_B; \mathcal{D}) = 0.30$

$$L(\theta_A; \mathcal{D}) > L(\theta_B; \mathcal{D})$$

θ_A makes the observed label $y^\star = 1$ less surprising.

likelihood of the fixed observation



Likelihood is a **score as a function of θ**; the scores need not sum to 1 over parameter settings.

Coin flips: a likelihood you can compute

D DERIVATION

Observed: $H, H, T, T, T, H, H, T, T, T$

Hence $n_H = 4$ and $n_T = 6$.

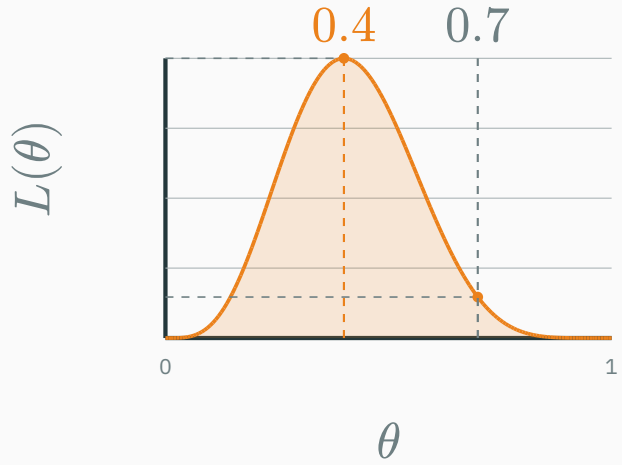
Model: $p(H \mid \theta) = \theta$, $p(T \mid \theta) = 1 - \theta$.

Independence gives

$$L(\theta) = \prod_i p(x_i \mid \theta)$$

$$= \theta^{n_H}(1-\theta)^{n_T}$$

$$= \theta^4(1-\theta)^6$$



$$L(0.4) \approx 0.00119$$
$$L(0.7) \approx 0.000175$$

about $6.8\times$ larger at $\theta = 0.4$

The likelihood peaks at $\theta = 0.4$ — the observed head fraction $\frac{4}{10}$.

Coin flips: calculate the log-likelihood

D DERIVATION

The **log-likelihood** is the logarithm of the likelihood — not a different model:

$$\ell(\theta) := \log L(\theta) = \log\left[\theta^4(1-\theta)^6\right] = 4\log\theta + 6\log(1-\theta)$$

Candidate $\theta = 0.4$

$L(0.4) \approx 0.00119$

$\ell(0.4) = 4\log 0.4 + 6\log 0.6 \approx -6.73$

Candidate $\theta = 0.7$

$L(0.7) \approx 0.000175$

$\ell(0.7) = 4\log 0.7 + 6\log 0.3 \approx -8.65$

Likelihoods below 1 have negative logs; among them, -6.73 is the larger value.

The observed sequence has higher log-likelihood at $\theta = 0.4$.

Why logs? The maximizing parameter does not change

D DERIVATION

The logarithm is **strictly increasing** on positive numbers:

$$a > b > 0 \quad \Longrightarrow \quad \log a > \log b$$

Therefore it preserves the ordering of all positive likelihood values:

$$L(0.4) > L(0.7)$$

apply the increasing function $\log$

$$\log L(0.4) > \log L(0.7)$$

$$\arg\max_{\theta} L(\theta) = \arg\max_{\theta} \log L(\theta)$$

Both likelihood and log-likelihood rank $\theta = 0.4$ above $\theta = 0.7$.

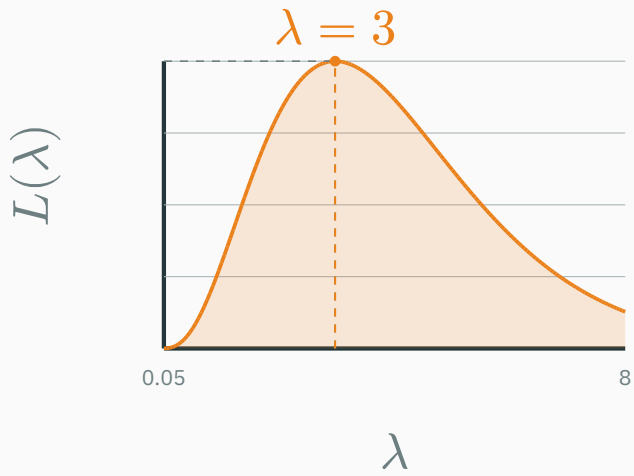
Poisson example: logging preserves the peak

V VISUAL

Suppose $Y \sim \text{Poisson}(\lambda)$ and the observed count is $y = 3$.

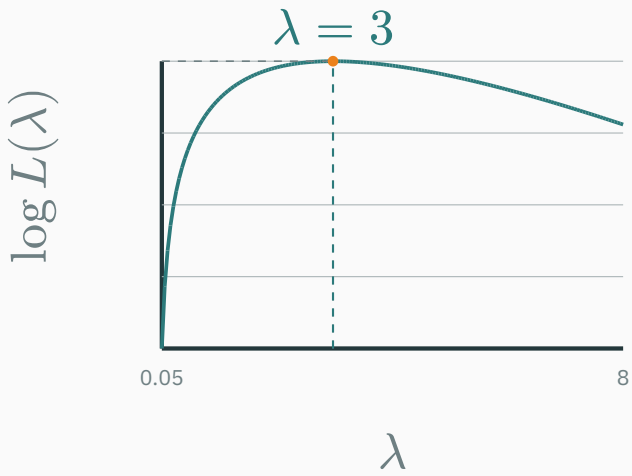
LIKELIHOOD

$$L(\lambda) = e^{-\lambda}\frac{\lambda^3}{3!}$$



LOG-LIKELIHOOD

$$\log L(\lambda) = 3\log\lambda - \lambda - \log 3!$$



Both curves peak at $\hat{\lambda}_{\text{MLE}} = 3$: the log changes the vertical scale, not the maximizing parameter.

Why logs? Products become stable sums

D DERIVATION

For independent observations,

$$L(\boldsymbol{\theta}) = \prod_i p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) \quad \Longrightarrow \quad \log L(\boldsymbol{\theta}) = \sum_i \log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i)$$

Worked numerical example. Suppose 200 observed outcomes each receive probability 0.01.

Direct product	$L = 0.01^{200} = 10^{-400} < 4.94 \times 10^{-324}$ → underflows to 0
Log space	$\log L = 200 \log(0.01) \approx -921.03$ → finite and representable

Float64 range: $\pm 1.80 \times 10^{308}$ · smallest positive normal 2.23×10^{-308} · smallest subnormal 4.94×10^{-324}.

Logs make optimization easier and prevent numerical underflow.

Coin flips: solve for the stationary point

D DERIVATION

For $L(\theta) = \theta^{n_H}(1-\theta)^{n_T}$, the log-likelihood is

$$\log L(\theta) = n_H \log \theta + n_T \log(1-\theta)$$

Set the derivative to zero:

$$\frac{\mathrm{d}}{\mathrm{d}\theta} \log L(\theta) = \frac{n_H}{\theta} - \frac{n_T}{1-\theta} = 0$$

$$\Longrightarrow \theta^\star = \frac{n_H}{n_H + n_T} = \frac{4}{10} = 0.4$$

$\theta^\star = 0.4$ **is a candidate optimum. Next, check the curvature.**

The second derivative confirms a maximum

D DERIVATION

Differentiate the log-likelihood again:

$$\frac{\mathrm{d}^2}{\mathrm{d}\theta^2} \log L(\theta) = -\frac{n_H}{\theta^2} - \frac{n_T}{(1-\theta)^2}$$

For $n_H, n_T > 0$ and $0 < \theta < 1$, both terms are negative:

$$\frac{\mathrm{d}^2}{\mathrm{d}\theta^2} \log L(\theta) < 0$$

At the stationary point $\theta^\star = 0.4$,

$$-\frac{4}{(0.4)^2} - \frac{6}{(0.6)^2} = -25 - 16.67 \approx -41.67 < 0$$

Strict concavity + stationary point → unique global maximum: $\hat{\theta}_{\mathrm{MLE}} = \theta^\star = \frac{4}{10} = 0.4$**.**

Maximum likelihood estimation

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg\max_{\boldsymbol{\theta}} p(\mathcal{D} \mid \boldsymbol{\theta})$$

Interpretation. Choose the parameter values that assign the **highest likelihood** to the observed dataset.

Maximizing likelihood = minimizing NLL

D DERIVATION

$$\arg\max_{\boldsymbol{\theta}} \prod_i p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = \arg\max_{\boldsymbol{\theta}} \sum_i \log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i)$$

$$= \arg\min_{\boldsymbol{\theta}} \left(-\sum_i \log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) \right)$$

$$\ell_{i(\boldsymbol{\theta})} = -\log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i)$$

Empirical risk = average training loss

D DERIVATION

In ML, **risk** means expected loss. Because the data-generating distribution is unknown, we estimate it from the observed training set.

Empirical risk = average loss on $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$.

Per example	$\ell_{i(\boldsymbol{\theta})} = -\log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i)$
Training set	$\hat{R}(\boldsymbol{\theta}) = \frac{1}{n}\sum_i \ell_{i(\boldsymbol{\theta})} = -\frac{1}{n}\log p(\mathcal{D} \mid \boldsymbol{\theta})$

The final equality uses conditional independence: $p(\mathcal{D}|\boldsymbol{\theta}) = \prod_i p_{\boldsymbol{\theta}}(y_i|\boldsymbol{x}_i)$.

Minimizing empirical NLL is maximum likelihood; $\frac{1}{n}$ only rescales the objective.

Worked example: support becomes a hard loss

D DERIVATION

Recall the Uniform observation model:

$$p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = \begin{cases} 1/(b-a) & a \le y_i \le b \\ 0 & \text{otherwise} \end{cases}$$

If an observed target lies outside its support, then

$$p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = 0$$

Its contribution to the negative log-likelihood is

$$\ell_{i(\boldsymbol{\theta})} = -\log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = -\log 0 = +\infty$$

Zero likelihood becomes an infinite training penalty.

Checkpoint: uniform support

Q QUESTION

For a Uniform(a, b) observation model, what is the NLL if the target lies outside $[a, b]$?

A. 0

B. A fixed finite penalty

C. $-\log 0 = +\infty$

D. It depends only on σ

Answer: uniform support

A ANSWER

Correct: C — $-\log 0 = +\infty$

Why: The model assigns zero density outside its support, so the observation is impossible under the assumption.

Regression: why MSE?

Regression as signal plus noise

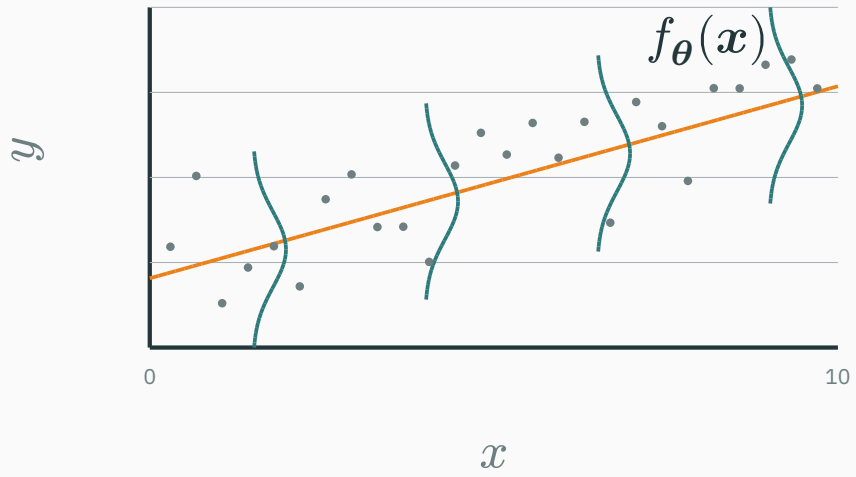
$$y_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Therefore the conditional output distribution is

$$Y_i \mid \boldsymbol{X}_i = \boldsymbol{x}_i \sim \mathcal{N}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \sigma^2)$$

What the assumption means visually

V VISUAL



Gaussian noise lives in the **output direction**, conditional on x — a vertical bell at every input.

Gaussian likelihood

D DERIVATION

$$p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\left(y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)^2}{2\sigma^2} \right)$$

Negative log of a single example:

$$-\log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i) = \frac{\left(y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)^2}{2\sigma^2} + C$$

MSE is Gaussian MLE

Sum over the dataset:

$$-\log p(\mathcal{D} \mid \boldsymbol{\theta}) = \frac{1}{2\sigma^2} \sum_i \left(y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)^2 + C$$

With σ fixed, the minimizer is

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg\min_{\boldsymbol{\theta}} \sum_i \left(y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)^2$$

MSE = Gaussian NLL, up to constants.

Linear regression makes f_θ explicit

D DERIVATION

For the one-dimensional picture,

$$\hat{y}_i = \theta_0 + \theta_1 x_i$$

For d features, prepend a constant 1 so the intercept is part of the same parameter vector:

$$\tilde{\boldsymbol{x}}_i = (1, \boldsymbol{x}_i), \quad \boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d)$$

Then the same notation used throughout the lecture becomes

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) = \boldsymbol{\theta}^\top \tilde{\boldsymbol{x}}_i$$

θ_0 is the intercept; $\theta_1, \ldots, \theta_d$ are the slopes.

Gaussian MLE becomes ordinary least squares

D DERIVATION

Substitute the linear mean into the observation model:

$$Y_i \mid \boldsymbol{X}_i = \boldsymbol{x}_i \sim \mathcal{N}(\boldsymbol{\theta}^\top \tilde{\boldsymbol{x}}_i, \sigma^2)$$

The vertical residual of point i is

$$r_i = y_i - \hat{y}_i = y_i - \boldsymbol{\theta}^\top \tilde{\boldsymbol{x}}_i$$

With σ fixed, maximum likelihood therefore gives

$$\hat{\boldsymbol{\theta}}_{\mathrm{MLE}} = \arg\min_{\boldsymbol{\theta}} \sum_i \underbrace{r_i^2}_{\text{squared vertical error}}$$

Ordinary least squares is Gaussian maximum likelihood for a linear mean.

The normal equations solve linear regression

D DERIVATION

Stack the augmented inputs into $\tilde{\boldsymbol{X}} = [\mathbf{1} \quad \boldsymbol{X}]$. Then

$$\hat{\boldsymbol{y}} = \tilde{\boldsymbol{X}}\boldsymbol{\theta}, \quad J(\boldsymbol{\theta}) = \left\|\boldsymbol{y} - \tilde{\boldsymbol{X}}\boldsymbol{\theta}\right\|_2^2$$

Differentiate and set the gradient to zero:

$$\nabla_{\boldsymbol{\theta}} J = 2\tilde{\boldsymbol{X}}^\top\left(\tilde{\boldsymbol{X}}\boldsymbol{\theta} - \boldsymbol{y}\right) = \mathbf{0}$$

This gives the **normal equations**:

$$\tilde{\boldsymbol{X}}^\top\tilde{\boldsymbol{X}}\hat{\boldsymbol{\theta}} = \tilde{\boldsymbol{X}}^\top\boldsymbol{y}$$

If $\tilde{\boldsymbol{X}}^\top\tilde{\boldsymbol{X}}$ is invertible,

$$\hat{\boldsymbol{\theta}} = \left(\tilde{\boldsymbol{X}}^\top\tilde{\boldsymbol{X}}\right)^{-1}\tilde{\boldsymbol{X}}^\top\boldsymbol{y}$$

Linear regression has a closed form; neural networks keep the MSE objective but use iterative optimization.

What role does σ^2 play?

★ OPTIONAL

$$\ell_i = \frac{(y_i - \mu_i)^2}{2\sigma^2} + \frac{1}{2}\log \sigma^2 + C$$

- one global σ^2 for every example → **homoscedastic** regression
- small σ: errors penalized **strongly**
- large σ: errors deemed **less surprising**

With fixed σ^2, every input has the same predicted noise width.

Homoscedastic vs input-dependent uncertainty

V VISUAL

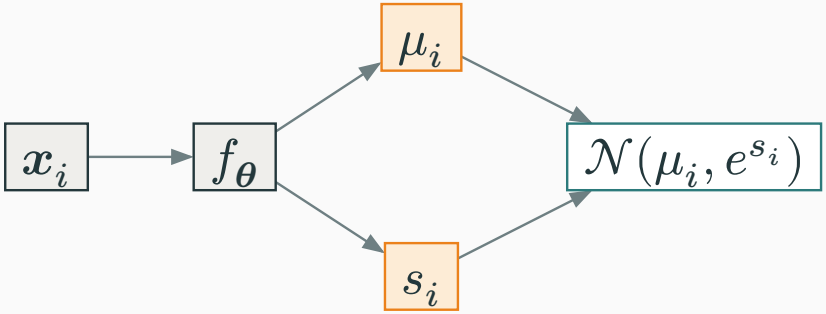
HOMOSCEDASTIC

Network output: μ_i

Shared variance: σ^2

same uncertainty at every x_i

HETEROSCEDASTIC



Network outputs: μ_i and s_i

$$s_i = \log \sigma_i^2, \quad \sigma_i^2 = e^{s_i} > 0$$

uncertainty changes with x_i

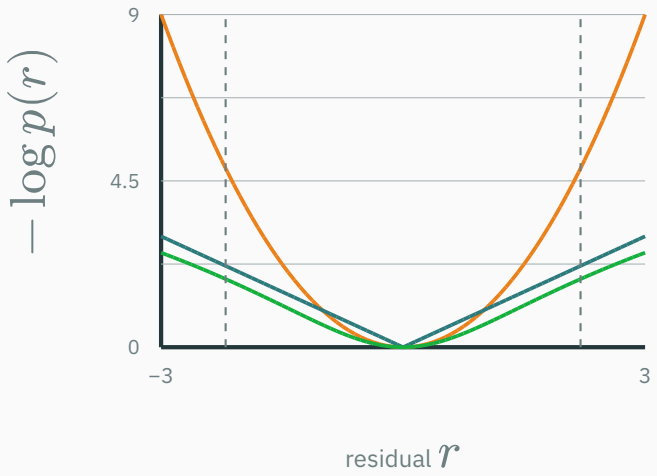
Predict log-variance so every real s_i maps to a positive variance e^{s_i}.

Different noise, different loss

V VISUAL

Noise model	Negative log-likelihood
Gaussian	r^2
Laplace	$\lvert r \rvert$
Uniform	0 inside; $+\infty$ outside
Student-t	$\log\left(1 + \frac{r^2}{\nu}\right)$

$r = y - \hat{y}$ · swatches match plot



Interactive: likelihood → loss

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/likelihood-to-loss

likelihood-to-loss.html — three synchronized panels: the **noise density** $p(r)$, the **loss** $-\log p(r)$, and **data with a fitted line**.
Controls: Gaussian / Laplace / Uniform / Student-t · noise scale · slope & intercept · **add an outlier** · per-point contributions.

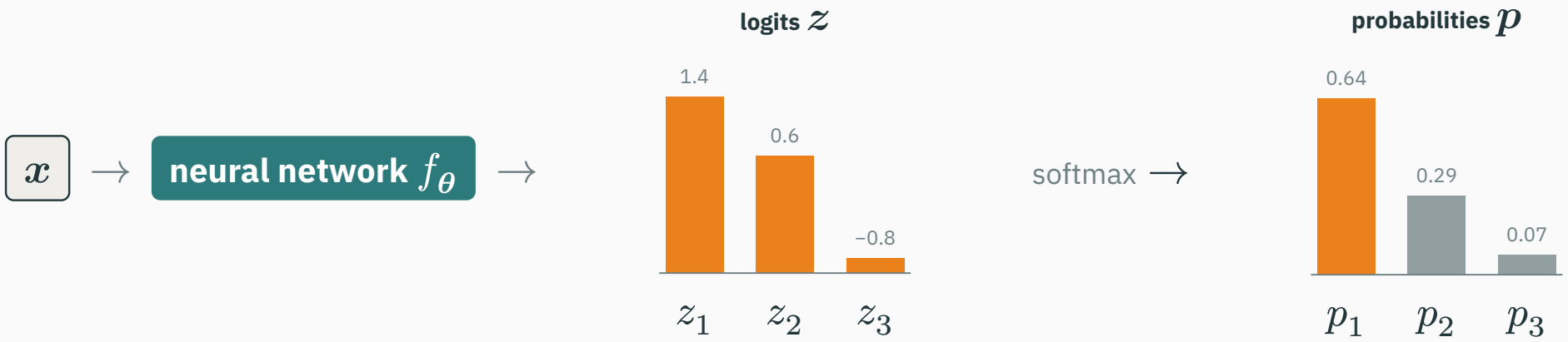
Ask: which model reacts most to an outlier? why does Uniform give an infinite barrier? which loss is robust?

Classification: cross-entropy

Classification should output probabilities

For K classes:

$$p_{\boldsymbol{\theta}}(Y = k \mid \boldsymbol{x}) \geq 0, \qquad \sum_{k=1}^{K} p_{\boldsymbol{\theta}}(Y = k \mid \boldsymbol{x}) = 1$$



logits $z_k \in \mathbb{R} \rightarrow$ **probabilities** $p_k \in [0, 1]$.

Binary classification: Bernoulli model

$$Y \mid \boldsymbol{X} = \boldsymbol{x} \sim \text{Bernoulli}(p_{\boldsymbol{\theta}}(\boldsymbol{x}))$$

Compact one-line form:

$$p_{\boldsymbol{\theta}}(y \mid \boldsymbol{x}) = p^y (1-p)^{1-y}$$

$y = 1 \Rightarrow p(y) = p$

$y = 0 \Rightarrow p(y) = 1 - p$

Bernoulli NLL gives BCE

D DERIVATION

$$-\log p_{\boldsymbol{\theta}}(y \mid \boldsymbol{x}) = -\log\big(p^{y}(1-p)^{1-y}\big)$$

$$= -y\log p - (1-y)\log(1-p)$$

Binary cross-entropy = Bernoulli NLL.

Check. True label $y = 1$: if $p = 0.8$ the loss is $-\log 0.8 \approx 0.22$; if $p = 0.2$ it is $-\log 0.2 \approx 1.61$.

Why use logits?

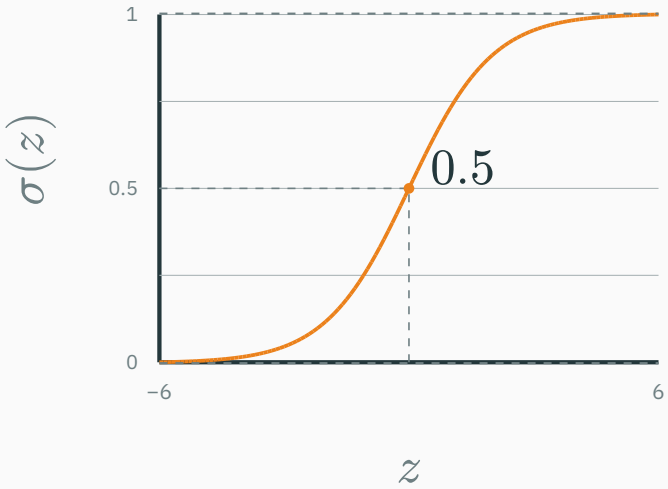
V VISUAL

The network emits an unbounded score $z = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \in \mathbb{R}$; the **sigmoid** maps it to a probability:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z \to -\infty : p \to 0$$
$$z = 0 : p = 0.5$$
$$z \to +\infty : p \to 1$$

In practice sigmoid + BCE are fused for numerical stability.



Logistic regression = Bernoulli MLE

D DERIVATION

Use an augmented input $\tilde{\boldsymbol{x}}_i$ so $\boldsymbol{\theta}$ contains both weights and the intercept:

$$p_i = \sigma(\boldsymbol{\theta}^\top \tilde{\boldsymbol{x}}_i), \quad Y_i \mid \boldsymbol{X}_i = \boldsymbol{x}_i \sim \text{Bernoulli}(p_i)$$

. Likelihood of the whole labelled dataset (i.i.d.):

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}$$

Its negative log — the objective we minimise:

$$J(\boldsymbol{\theta}) = -\sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Logistic regression is Bernoulli maximum likelihood; its training objective is **binary cross-entropy**.

Multi-class model and softmax

V VISUAL

CATEGORICAL OBSERVATION

$$Y \mid \boldsymbol{X} = \boldsymbol{x} \sim \text{Categorical}(p_1, \ldots, p_K)$$

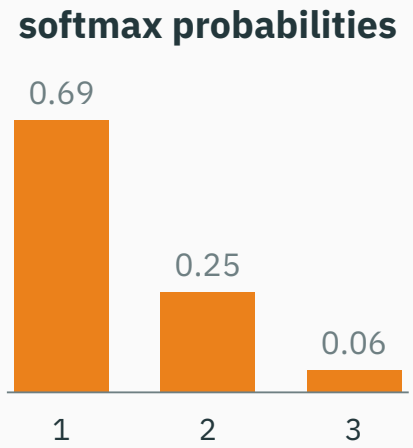
A **one-hot** target has one 1 at the true class and 0 everywhere else.

Example: class 2 of 3 $\rightarrow \boldsymbol{y} = (0, 1, 0)$

$$p_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_{k=1}^{K} p_k^{y_k}$$

NETWORK PARAMETERIZATION

$$\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x}), \quad p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$



Softmax guarantees $p_k > 0$ and $\sum_k p_k = 1$, as a categorical model requires.

Categorical NLL gives cross-entropy

D DERIVATION

$$-\log p_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \boldsymbol{x}) = -\log \prod_k p_k^{y_k} = -\sum_k y_k \log p_k$$

Because $\boldsymbol{y}$ is one-hot, only the true class survives:

$$\mathcal{L} = -\log p_{\text{true class}}$$

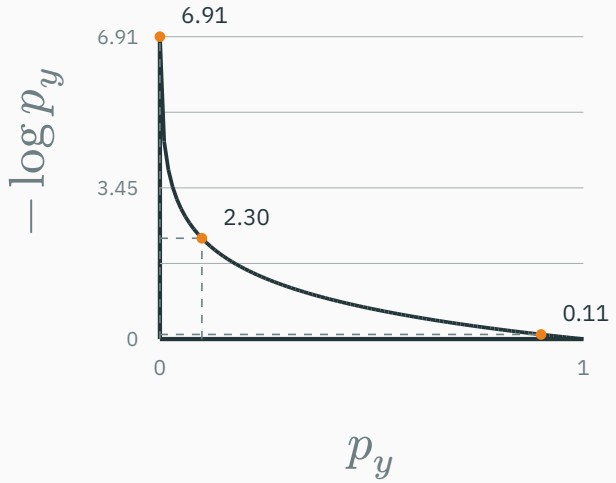
Confident mistakes are expensive

V VISUAL

$$\ell(p_y) = -\log p_y$$

Read off the loss at three confidences for the **true** class:

$p_y = 0.9 \Rightarrow \ell \approx 0.105$
$p_y = 0.1 \Rightarrow \ell \approx 2.303$
$p_y = 0.001 \Rightarrow \ell \approx 6.908$



The negative log makes a confident wrong prediction a large loss.

Checkpoint: confident mistakes

Q QUESTION

Why does cross-entropy penalize a confidently wrong prediction strongly?

A. It makes all class probabilities equal

B. The true-class probability is near zero, so $-\log p_y$ is large

C. It adds an L_2 penalty to the logits

D. The gradient is always zero

Answer: confident mistakes

A ANSWER

Correct: B — The true-class probability is near zero

Why: The negative log grows without bound as p_y approaches zero, matching the fact that the model declared the observed class nearly impossible.

Interactive: logits → softmax → cross-entropy

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/softmax-cross-entropy

softmax-cross-entropy.html — controls z_1, z_2, z_3, the true class, and temperature T; presets for **correct & confident / correct & unsure / wrong & unsure / wrong & confident**. Shows $z \to \text{softmax}(z/T) \to -\log p_y$.

Ask students to predict the loss before it is revealed.

Why not MSE for classification?

★ OPTIONAL

MSE is not *impossible*, but it usually corresponds to a **mismatched observation model** for class indicators.

Gaussian view $Y_k = p_k + \varepsilon_k, \varepsilon_k \sim \mathcal{N}(0, \sigma^2)$

Categorical view $Y \sim \text{Categorical}(\boldsymbol{p})$

Cross-entropy also gives **stronger gradients** on confident errors (where MSE's gradient vanishes).

Bayes’ rule for learning

Bayes' rule: from events to model parameters

V VISUAL

PROBABILITY LECTURE

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

A — event or hypothesis to infer
B — observed evidence

Examples: disease given a test; urn given a sampled colour.

MACHINE / DEEP LEARNING

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}$$

$\boldsymbol{\theta}$ — model parameters to infer
$\mathcal{D}$ — observed training dataset

Question: which parameter settings remain plausible after seeing the data?

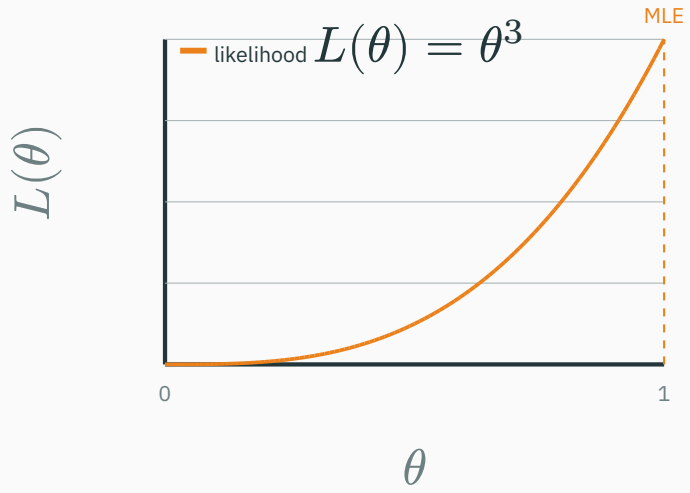
Replace A by θ and B by $\mathcal{D}$: the algebra is unchanged; the interpretation changes.

Three heads do not prove $\theta = 1$

Q QUESTION

Let $\theta = P(H)$ and suppose the first three flips are $\mathcal{D} = (H, H, H)$.

H · H · H



$$L(\theta; \mathcal{D}) = p(\mathcal{D} \mid \theta) = \theta^3$$

$$\hat{\theta}_{\text{MLE}} = \arg\max_{0 \le \theta \le 1} \theta^3 = 1$$

Did we learn that the coin **always** lands heads—or did a merely head-biased coin produce a lucky run?

MLE identifies the best-fitting θ; it does not make $\theta = 1$ certain.

A prior tempers the conclusion from three flips

V VISUAL

Take a mild prior centred on a fair coin: $\theta \sim \mathrm{Beta}(2, 2)$.

Prior

$p(\theta) = 6\theta(1-\theta)$

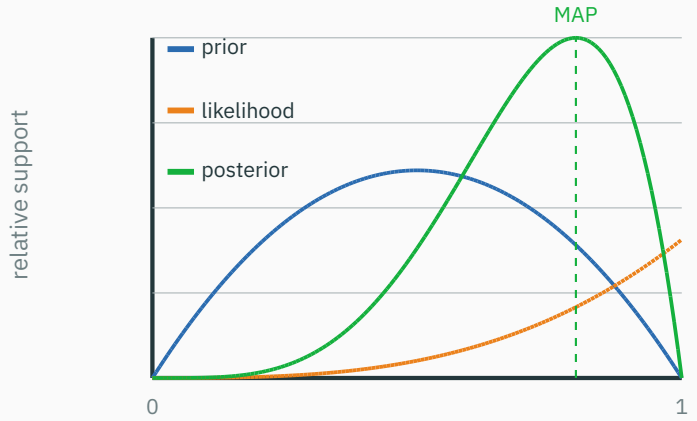
Likelihood of H, H, H

$p(\mathcal{D} \mid \theta) = \theta^3$

Posterior

$$p(\theta \mid \mathcal{D}) \propto \theta^3 p(\theta)$$
$$= \mathrm{Beta}(5, 2)$$

$\hat{\theta}_{\mathrm{MLE}} = 1, \quad \hat{\theta}_{\mathrm{MAP}} = \frac{4}{5} = 0.8$



The posterior moves toward heads, but remains spread over many possible coins.

With little data, the prior matters; as observations accumulate, the likelihood increasingly dominates.

Priors, MAP and regularization

Every learner has an inductive bias

V VISUAL

When several predictors fit the training data, the learner still needs a preference for **unseen inputs**. That preference is its **inductive bias**.

Model	Built-in preference → consequence
Linear regression	responses vary linearly with features → line or plane extrapolation
k-NN	nearby inputs tend to have similar outputs → local prediction
Decision tree	a few axis-aligned rules partition the space → piecewise-constant regions
CNN	useful local patterns repeat across positions → shared filters recognize translations

A prior is one explicit inductive bias; the model class, features, architecture, and optimization add others.

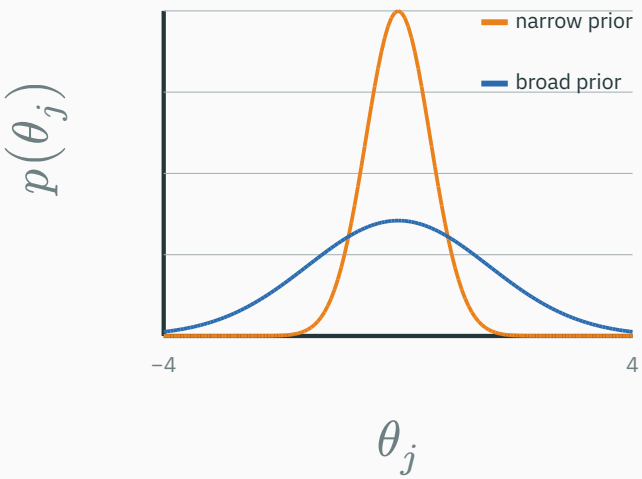
A neural-network prior ranks plausible parameter vectors

V VISUAL

$$\boldsymbol{\theta} \sim p(\boldsymbol{\theta})$$

For a neural network, $\boldsymbol{\theta}$ contains all weights and biases. Before using the current dataset, a prior states which settings are more plausible.

It can prefer small or sparse weights, smooth functions, and correlated parameters.



narrow = stronger preference near 0

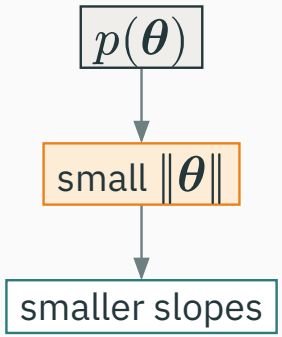
A prior makes part of the model's inductive bias explicit as a probability distribution.

Parameter priors have visible consequences

V VISUAL

REGRESSION

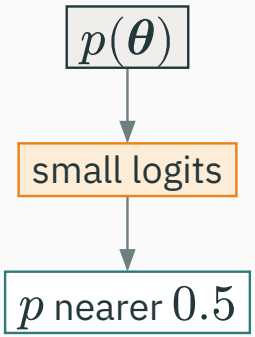
$$Y_i \mid \boldsymbol{X}_i = \boldsymbol{x}_i \sim \mathcal{N}(\boldsymbol{\theta}^\top \tilde{\boldsymbol{x}}_i, \sigma^2)$$



Before seeing data, flatter linear predictions are more plausible.

BINARY CLASSIFICATION

$$p_{\boldsymbol{\theta}}(Y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{\theta}^\top \tilde{\boldsymbol{x}})$$



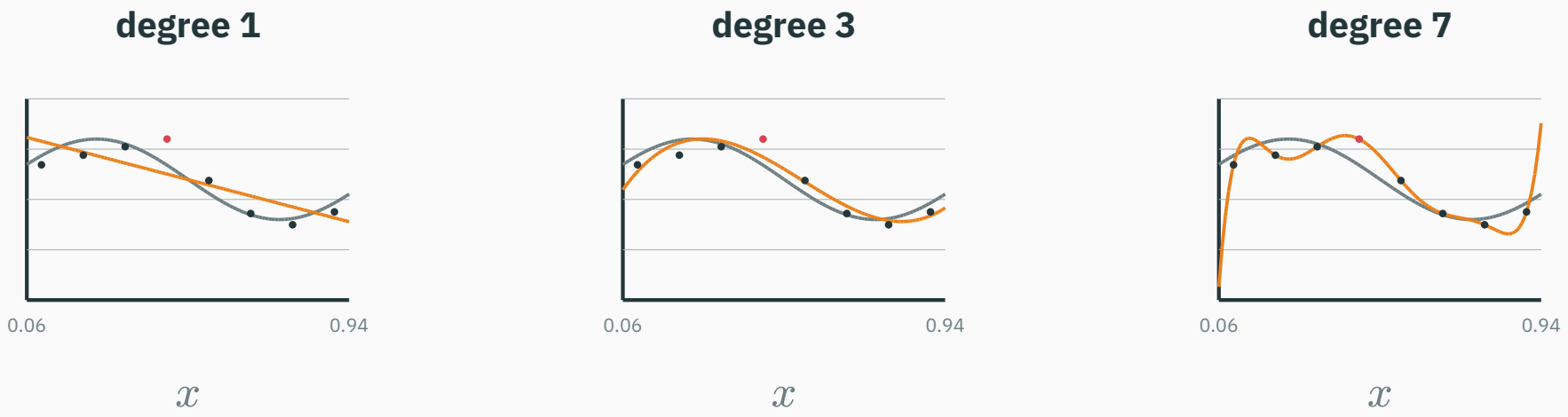
Before seeing data, extremely confident predictions are less plausible.

The prior acts on parameters; its visible effect depends on how those parameters enter the model.

Model complexity: underfitting to overfitting

Fit the same eight observations by least squares; only the polynomial degree changes.

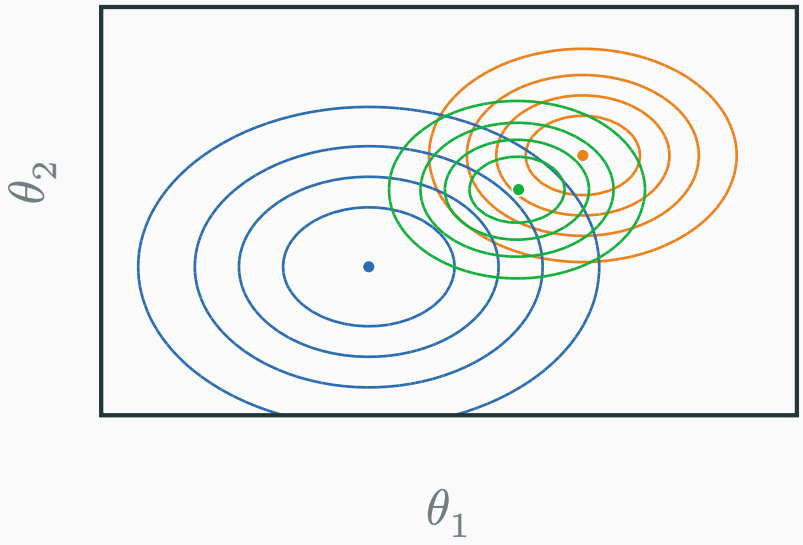
dashed = underlying trend · orange = fitted polynomial · red dot = one noisy observation



Higher degree lowers training error, but degree 7 chases the noisy observation and becomes unstable. Priors and regularization can prefer smoother solutions.

Bayes' rule over parameters

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})}$$



Contours: **likelihood** **prior** **posterior**

● MLE ● MAP ● prior mean

The posterior combines likelihood and prior; $p(\mathcal{D})$ normalizes it, and its mode is the MAP.

MAP estimation

D DERIVATION

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D}) = \arg\max_{\boldsymbol{\theta}} p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})$$

Take negative logs:

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \arg\min_{\boldsymbol{\theta}} \Bigg(\underbrace{-\log p(\mathcal{D} \mid \boldsymbol{\theta})}_{\text{NLL}} + \underbrace{-\log p(\boldsymbol{\theta})}_{\text{negative log-prior}} \Bigg)$$

MAP objective: NLL + negative log-prior

In ML language: data loss + regularization.

A Gaussian prior is L_2 regularization

D DERIVATION

Assume the parameters are centred at zero before seeing the data:

$\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \tau^2 \boldsymbol{I})$.

$$\underbrace{-\log p(\mathcal{D} \mid \boldsymbol{\theta})}_{\text{NLL}} + \underbrace{-\log p(\boldsymbol{\theta})}_{\text{negative log-prior}}$$

$$-\log p(\boldsymbol{\theta}) = \frac{1}{2\tau^2}\|\boldsymbol{\theta}\|_2^2 + C$$

$$\Rightarrow \quad \mathcal{L}_{\text{MAP}} = \text{NLL} + \underbrace{\frac{1}{2\tau^2}}_{\lambda}\|\boldsymbol{\theta}\|_2^2 + C$$

$\lambda = \frac{1}{2\tau^2}$: smaller $\tau \to$ larger $\lambda \to$ stronger shrinkage toward 0.

Prior width controls how far MAP moves

V VISUAL

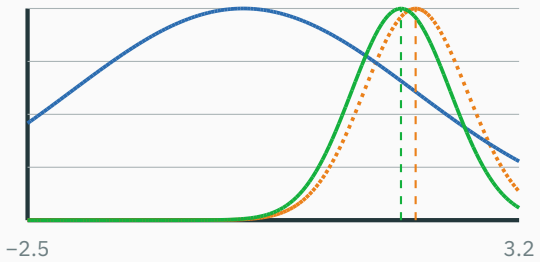
Same likelihood: $L(\theta) \propto \mathcal{N}\left(\theta; \hat{\theta}_{\text{MLE}} = 2, s^2\right)$ with $s = 0.6$; prior: $p(\theta) = \mathcal{N}(0, \tau^2)$.

$$\hat{\theta}_{\text{MAP}} = \frac{\tau^2}{\tau^2 + s^2}\hat{\theta}_{\text{MLE}}$$ (a shrinkage factor between 0 and 1)

··· likelihood – – prior — posterior

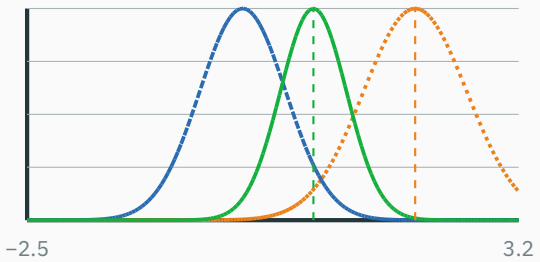
Broad prior: $\tau = 2, \lambda = 0.125$

$\text{MLE} = 2 \rightarrow \text{MAP} \approx 1.83$



Narrow prior: $\tau = 0.5, \lambda = 2$

$\text{MLE} = 2 \rightarrow \text{MAP} \approx 0.82$

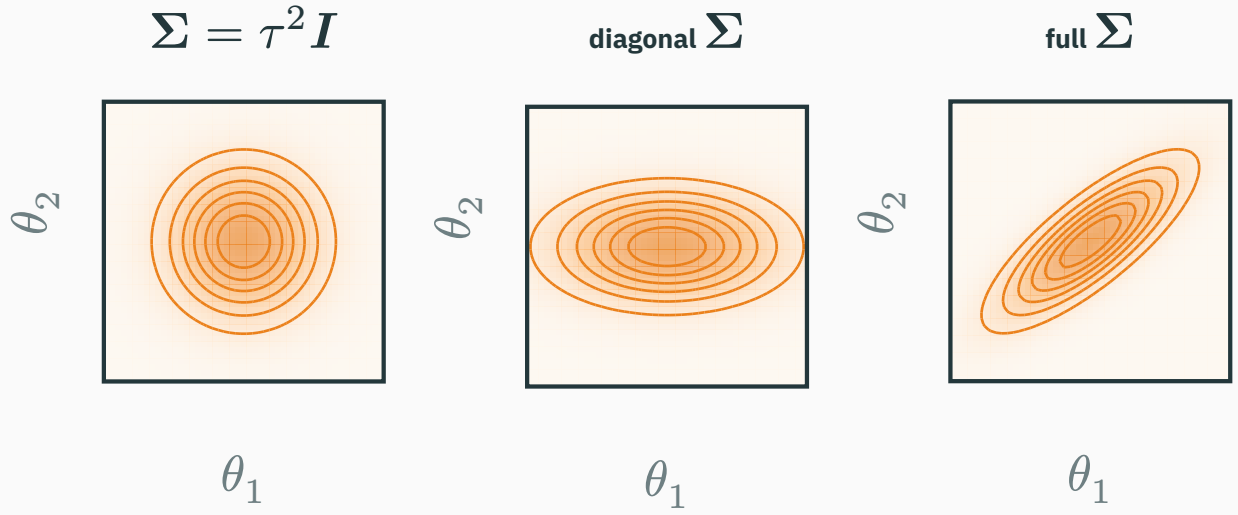


The narrower prior gives the larger λ and pulls MAP farther from the MLE toward zero.

A multivariate Gaussian prior has geometry

V VISUAL

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad -\log p(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{\mu}) + C$$



Axes: θ_1 and θ_2 are two coordinates of the parameter vector $\boldsymbol{\theta}$.

Colour = prior density $p(\boldsymbol{\theta})$: pale outer regions are less plausible; darker centres are more plausible.

Covariance says **which directions in parameter space are plausible.**

A Laplace prior is L_1 regularization

★ OPTIONAL

Assume the parameter components are independent, with

$$p(\theta_j) = \frac{1}{2b} \exp\left(-\frac{|\theta_j|}{b}\right).$$

$$p(\boldsymbol{\theta}) = \prod_j p(\theta_j).$$

Taking negative logs gives

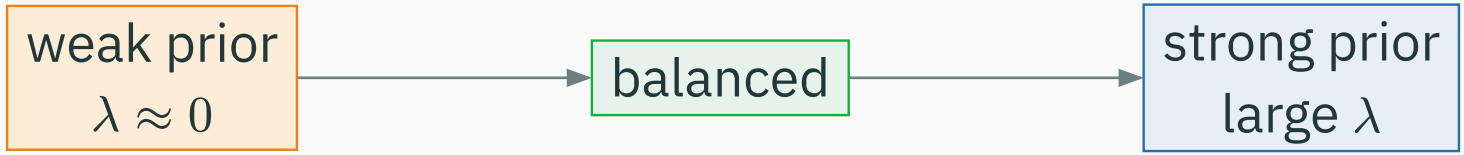
$$-\log p(\boldsymbol{\theta}) = \frac{1}{b}\sum_j |\theta_j| + C = \underbrace{\frac{1}{b}}_{\lambda} \|\boldsymbol{\theta}\|_1 + C.$$

$\lambda = \frac{1}{b}$: a narrower Laplace prior gives stronger L_1 regularization; its sharp peak at zero promotes sparsity.

Prior strength controls underfitting and overfitting

V VISUAL

For the summed-NLL convention used above, $\lambda = \frac{1}{2\tau^2}$:



DATA FIT DOMINATES

flexible model

high variance · overfit

LIKELIHOOD + PRIOR

enough flexibility

better generalization

PRIOR DOMINATES

weights forced small

high bias · underfit

Regularization trades variance for bias; more is not always better.

Interactive: likelihood, prior, MAP

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/map-regularization

map-regularization.html — a two-parameter linear model so contours are visible. Shows **likelihood**, **prior**, and **posterior** contours with the **MLE**, **MAP**, and zero points.
Controls: amount of data · noise · prior variance · Gaussian vs Laplace · prior correlation.

Ask: more data? narrower prior? why does MAP → MLE as data grows? a correlated prior?

Full Bayes and deep learning

MAP is still one parameter vector

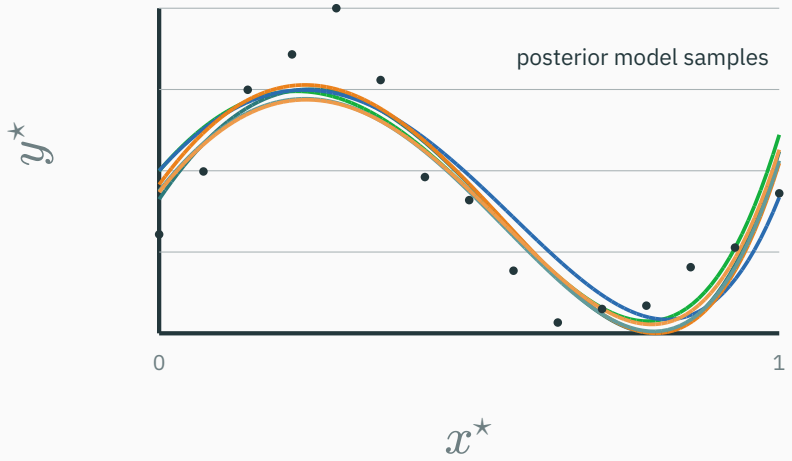
MLE $\hat{\theta}_{\mathrm{MLE}}$ — one point

MAP $\hat{\theta}_{\mathrm{MAP}}$ — one point

Full Bayes keeps the whole posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ — a **distribution**, not a point.

Posterior predictive distribution

$$p(y^\star \mid \boldsymbol{x}^\star, \mathcal{D}) = \int p_{\boldsymbol{\theta}}(y^\star \mid \boldsymbol{x}^\star)\, p(\boldsymbol{\theta} \mid \mathcal{D})\, \mathrm{d}\boldsymbol{\theta}$$



Bayesian prediction **averages over plausible models** — and reports uncertainty.

Why DL usually uses point estimates

★ OPTIONAL

A modern network has millions to billions of parameters, so integrating $p(\boldsymbol{\theta} \mid \mathcal{D})$ is hard. Common practical choices:

- MLE · MAP
- deep **ensembles**
- **dropout**-based approximations
- **variational** inference

The standard supervised DL objective

$$\min_{\boldsymbol{\theta}} \underbrace{\frac{1}{|B|} \sum_{i \in B} -\log p_{\boldsymbol{\theta}}(y_i \mid \boldsymbol{x}_i)}_{\text{likelihood / data fit}} + \underbrace{\lambda \|\boldsymbol{\theta}\|_2^2}_{\text{prior / regularization}}$$

With an averaged NLL, λ absorbs the dataset or minibatch scaling convention.

probabilistic model + neural network + gradient optimization

DL doesn't discard probabilistic ML — it **scales** it with expressive functions and gradient descent.

Retrieval summary

V VISUAL

Gaussian likelihood	$\Longrightarrow$ **MSE**
Categorical likelihood	$\Longrightarrow$ **cross-entropy**
Gaussian prior	$\Longrightarrow L_2$
Laplace prior	$\Longrightarrow L_1$
MLE + prior	$\Longrightarrow$ **MAP**

Every supervised objective begins with two assumptions

V VISUAL

MODEL

observation model

$$p_{\boldsymbol{\theta}}(y \mid \boldsymbol{x})$$

parameter prior

$$p(\boldsymbol{\theta})$$

$\rightarrow$

TRAIN

$$\mathcal{L}_{\mathrm{MAP}(\boldsymbol{\theta})} = \underbrace{\mathrm{NLL}(\mathcal{D}; \boldsymbol{\theta})}_{\text{data term}} + \underbrace{\Omega(\boldsymbol{\theta})}_{\text{parameter term}}$$

$$\Omega(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$$

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{MAP}}$$

$\rightarrow$

PREDICT

plug-in: fix $\boldsymbol{\theta} = \hat{\boldsymbol{\theta}}$

$$p_{\hat{\boldsymbol{\theta}}}(y^\star \mid \boldsymbol{x}^\star)$$

or average over the posterior

$$p(y^\star \mid \boldsymbol{x}^\star, \mathcal{D})$$

Loss $\leftrightarrow$ likelihood; regularizer $\leftrightarrow$ prior; prediction $\leftrightarrow$ plug-in or posterior predictive.

Next: from linear models to neural networks — **neurons, nonlinearities, and multilayer perceptrons.**